



UNIVERSITY OF
PORTSMOUTH

PROGRAMMING FOR DATA ANALYSIS AND AI

COURSEWORK – Item 1

**School of Computing
University of Portsmouth**

MODULE CODE: M33145

COORDINATOR: Dr ATEFEH KHAZAEI GHOOZHDI

STUDENT ID: UP2302047

DATE: 15th January 2025

Table of Contents

Task 1	4
Descriptive analytics	4
Basic Statistics of all Attributes:	5
Visualization graphs	13
Task 2	18
Data Preparation.....	18
Handling Outliers Using IQR Method	22
Label Encoding for Categorical Variables	24
Handling Class Imbalance Using Random Oversampling	26
Feature Scaling with Standardization.....	27
Task 3	28
Perform Classification	28
K Nearest Neighbors (KNN) Algorithm Model Evaluation.....	29
Support Vector Machine (SVM) Model Evaluation	31
Decision Tree Model Evaluation.....	33
XGBoost Model Evaluation.....	35
Task 4	38
Perform Regression.....	38
Splitting the Dataset for Regression.....	39
Splitting the Data into Training and Testing Sets for Regression	39
Standardizing the Data for Regression	39
Support Vector Regression (SVR) Model Tuning and Evaluation	39
Random Forest Regression Model Tuning and Evaluation.....	40
Cross-Validation for SVR and Random Forest Models	40
Model Performance and Cross-Validation Results.....	41
Comparison between SVR and Random Forest (RF):	41
TASK 5	43
Perform Clustering.....	43
K Means Clustering algorithm.....	44
Determining the Optimal Number of Clusters Using the Elbow Method	45
Visualizing Clustering Quality with Silhouette Score.....	46
Mean Shift Clustering algorithm	48
Comparison between K-Means Clustering and Mean Shift Clustering:	50

Task 1

Descriptive analytics

Basic Statistics of all Attributes:

```

RangeIndex: 5110 entries, 0 to 5109
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   id                    5110 non-null   int64  
 1   gender                5110 non-null   object  
 2   age                   5110 non-null   float64 
 3   hypertension          5110 non-null   int64  
 4   heart_disease         5110 non-null   int64  
 5   ever_married          5110 non-null   object  
 6   work_type             5110 non-null   object  
 7   Residence_type        5110 non-null   object  
 8   avg_glucose_level     5110 non-null   float64 
 9   bmi                   4909 non-null   float64 
10   smoking_status        5110 non-null   object  
11   stroke                5110 non-null   int64  
dtypes: float64(3), int64(4), object(5)
memory usage: 479.2+ KB

```

Attributes	Null Values	Not-Null Values	Data Types
ID	0	5110	Int
Gender	0	5110	Object
Age	0	5110	Float
Hypertension	0	5110	Int
Heart disease	0	5110	Int
Ever married	0	5110	Object
Work type	0	5110	Object
Residence type	0	5110	Object
Average glucose level	0	5110	Float
Body mass index	201	4909	Float
Smoking status	0	5110	Object
Stroke	0	5110	Int

The table contains 5110 records of various health-related attributes. The fields in the dataset include ID, gender, age, hypertension, heart disease, marital status, work type, residence type, average glucose level, body mass index-with 201 null values-smoking status, and stroke. All the attributes are complete except for the body mass index, which has 201 missing entries. For example, numerical values have been kept as integers and floats for the respective data type of these fields, whereas object-type fields are gender, marital status, work type, and smoking status.

Gender:

```
Attribute: gender
      Count  Percentage (%)
gender
Female    2994      58.590998
Male      2115      41.389432
Other         1       0.019569
```

Gender	Count	Percentage
Female	2994	58
Male	2115	41
Other	1	0

The gender breakdown of the dataset includes 2994 females, 58%; 2115 males, 41%; and 1 other, 0%, representing the breakdown of gender across the 5110 records in the table.

Age:

```
12-18      313
19-25      380
26-32      413
33-40      521
40-50      730
50-60      834
60+       1376
Name: count, dtype: int64
```

Total : 5110

Age Group	Count
12 – 18	313
19 – 25	380
26 – 32	413
33 – 40	521
40 – 50	730
50 – 60	834
60 +	1376

The data indicates an increasing trend in counts across age groups, with the highest representation in the 60+ category, comprising 1376 individuals, which may indicate that the dataset focuses on older populations.

Hypertension:

Attribute: hypertension

	Count	Percentage (%)
hypertension		
0	4612	90.254403
1	498	9.745597

Status	Count	Percentage
No Hypertension	4612	90
Hypertension	498	9

This would mean that 90% of the records, which is 4612 individuals, did not have hypertension, whereas 498, or 9%, were diagnosed to suffer from hypertension. Therefore, the greater part of this dataset represents non-hypertensive individuals.

Heart Disease:

Attribute: heart disease

	Count	Percentage (%)
heart_disease		
0	4834	94.598826
1	276	5.401174

Status	Count	Percentage
No Heart Disease	4834	94
Heart Disease	276	5

This table indicates that 4834 (94%) of the studied individuals do not have heart disease, while 276 (5%) are those diagnosed with heart disease. This indicates a far larger amount of people not having the condition.

Ever Married:

```
Attribute: Ever married
      Count  Percentage (%)
ever_married
Yes          3353      65.616438
No           1757      34.383562
```

Ever Married	Count	Percentage
Yes	3353	65
No	1757	34

The marital status distribution is as follows: 3353 married, accounting for 65%, and 1757 never married, accounting for 34%. This therefore means a large majority of married individuals in the dataset compared to those who have not been married.

Work type:

```
Attribute: work type
      Count  Percentage (%)
work_type
Private      2925      57.240705
Self-employed  819      16.027397
children      687      13.444227
Govt_job      657      12.857143
Never_worked   22       0.430528
```

Work type	Count	Percentage
Private	2925	57
Self – employed	819	16
Children	687	13
Govt job	657	12
Never worked	22	0.43

Distribution by type of work reveals that the working group dominates this profile, at 57 percent, with a total of 2925 persons, followed by the self-employed at 16 percent with a total of 819 persons, and followed by children employed at 687 or 13 percent. This is followed by government jobs, 657 or 12 percent, and never worked, 22 persons, or 0.43 percent. This once more emphasizes the different occupational statuses across this data.

Residence type:

```
Attribute: Residence type
      Count  Percentage (%)
Residence_type
Urban      2596      50.802348
Rural      2514      49.197652
```

Residence type	Count	Percentage
Urban	2596	50
Rural	2514	49

Among all the residence type distributions, 2596 are living in an urban area whereas 2514 people are residing in a rural area, which accounts for 50% and 49%, respectively. It shows nearly equal proportions of respondents from both urban and rural locations.

Stroke:

```
Attribute: stroke
      Count  Percentage (%)
stroke
0        4861      95.127202
1         249       4.872798
```

Status	Count	Percentage
No Stroke	4861	95
Stroke	249	4

The dataset is imbalanced, having 95% of 5110 records for no stroke (4861) and 4% for stroke (249). This is a highly imbalanced distribution, with many more cases of no stroke than stroke. This may lead to poor performance in the model, since it may be biased toward the majority class, which is no stroke. This was handled by the oversampling technique, which involved artificially increasing the stroke cases to balance out the dataset. This ensures that both classes are of equal importance in the model for improved accuracy of the stroke case prediction.

Average glucose level:

```
count      5110.000000
mean       106.147677
std        45.283560
min        55.120000
25%        77.245000
50%        91.885000
75%       114.090000
max       271.740000
Name: avg_glucose_level, dtype: float64
```

Count	5110
Mean	106
Standard deviation	45
Minimum	55
25 %	77
50 %	91
75 %	114
Maximum	271

The table represents the statistical details of a particular variable in the dataset. The total count of entries is 5110. The mean value is 106, which indicates the average of the data. The standard deviation is 45, which reflects the variability or dispersion of the data. The minimum value is 55, while the maximum is 271, showing the range of the data. Q1, which is the 25th percentile, is 77; the median, or 50th percentile, is 91; and Q3, or the 75th percentile, is 114, showing the dispersion of the values within the data set.

Bmi:

```
count    4909.000000
mean      28.893237
std       7.854067
min       10.300000
25%       23.500000
50%       28.100000
75%       33.100000
max       97.600000
Name: bmi, dtype: float64
```

Count	4909
Mean	28
Standard deviation	7
Minimum	10
25 %	23
50 %	28
75 %	33
Maximum	97

The 'bmi' column within the data carries a total count of 4909 records. Also, the mean value of BMI is at 28 while the standard deviation is 7 to show some moderated dispersion from the mean. Further, the range is wide from minimum recorded values at 10-97 maximum recorded values. The 25th percentile (25%) is 23, the median (50%) is 28, and the 75th percentile (75%) is 33. This indicates that for the majority of the people, the BMI lies between 23 and 33.

Smoking status:

Attribute: smoking status

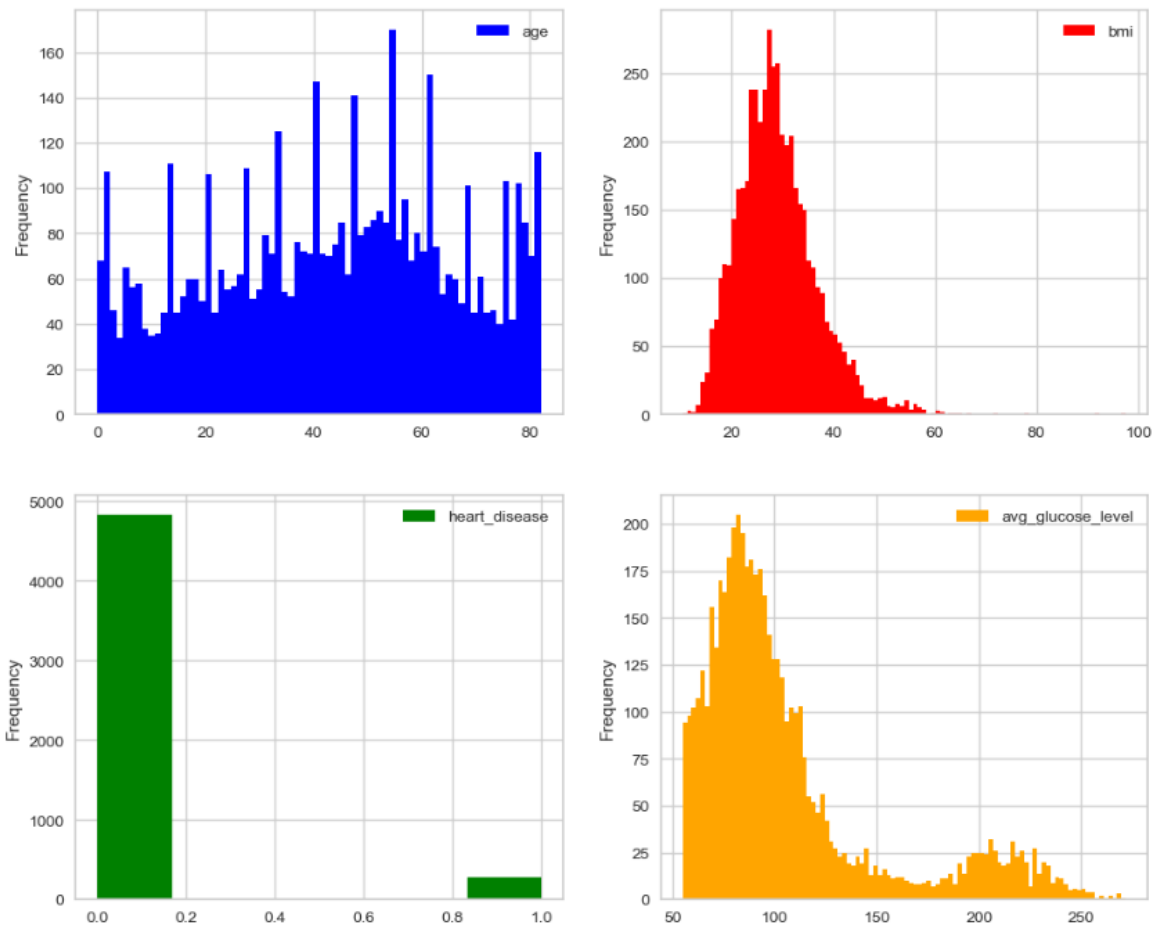
	Count	Percentage (%)
smoking_status		
never smoked	1892	37.025440
Unknown	1544	30.215264
formerly smoked	885	17.318982
smokes	789	15.440313

Smoking Status	Count	Percentage
Never smoked	1892	37
Unknown	1544	30
Formerly smoked	885	17
Smokes	789	15

The distribution of smoking status is that 1892 persons (37%) are never smoked, 1544 persons (30%) have a smoking status described as unknown, while 885 (17%) subjects are former smokers and 789 persons (15%) smoke now. Thus, this breakdown gives insight into the smoking habits that constitute quite a large portion for either persons that have never smoked or having their history of smoking unknown.

Visualization graphs

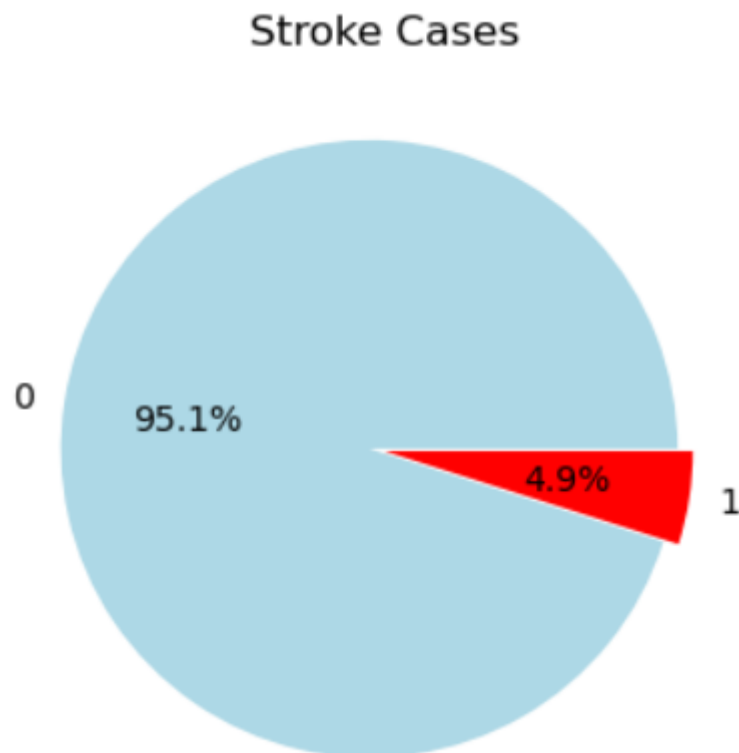
1) Histograms of Age, BMI, Heart Disease, and Avg Glucose Levels



- **Age:** Histogram with 70 bins showing the age distribution, highlighting range and clusters.
- **BMI:** Histogram with 100 bins providing detailed BMI variation and identifying anomalies.
- **Heart Disease:** Histogram with 6 bins depicting the presence (1) or absence (0) of heart disease, verifying its prevalence.

Average Glucose Levels: The last histogram maps the average of glucose levels using 100 bins. This plot is informative of the variation of glucose levels among participants and helps identify any trend or pattern that can be related to health conditions, such as diabetes.

2) Pie plot of Stroke Cases

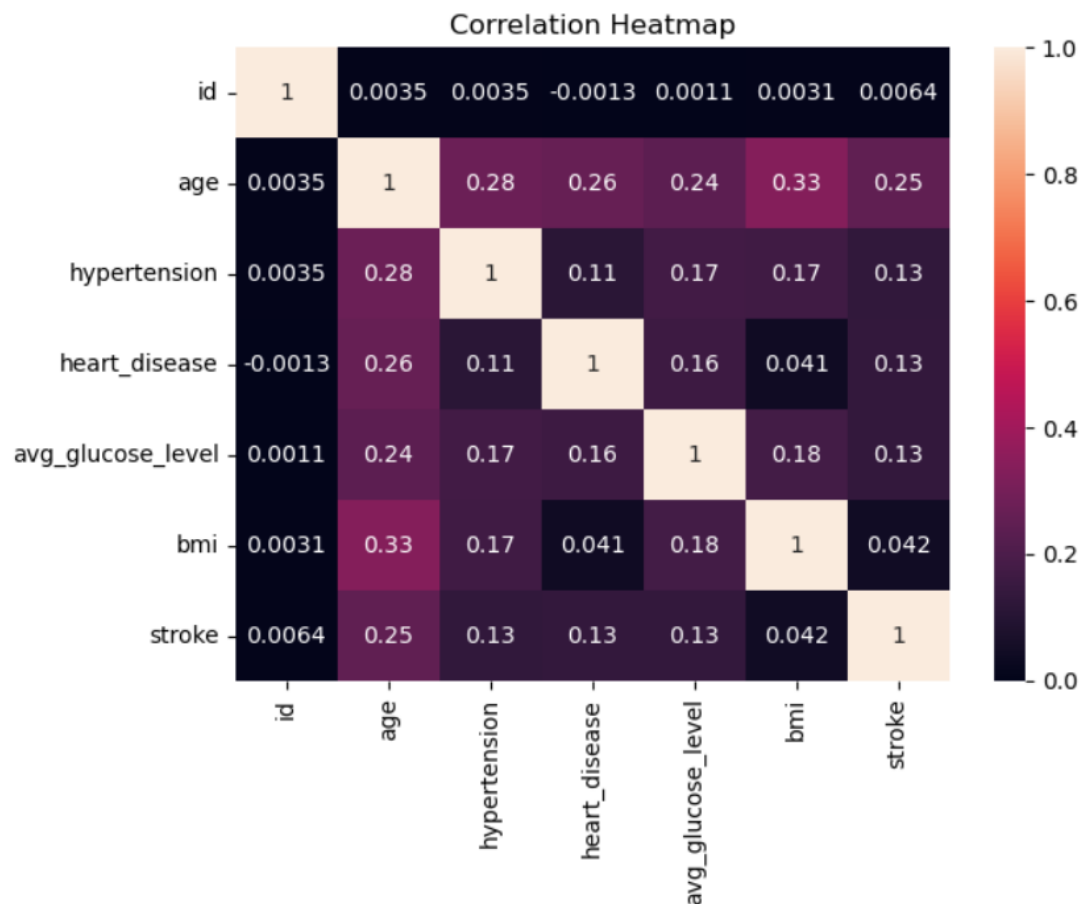


This pie chart represents the distribution of stroke cases in the dataset. It represents the proportion of individuals who have experienced a stroke versus those who have not:

- Stroke (1): The red slice represents the number of people who have had a stroke, which gives an overview of how many stroke cases are in the dataset.
- No Stroke (0): Light blue represents the subjects with no stroke and provides a comparison for the stroke cases.

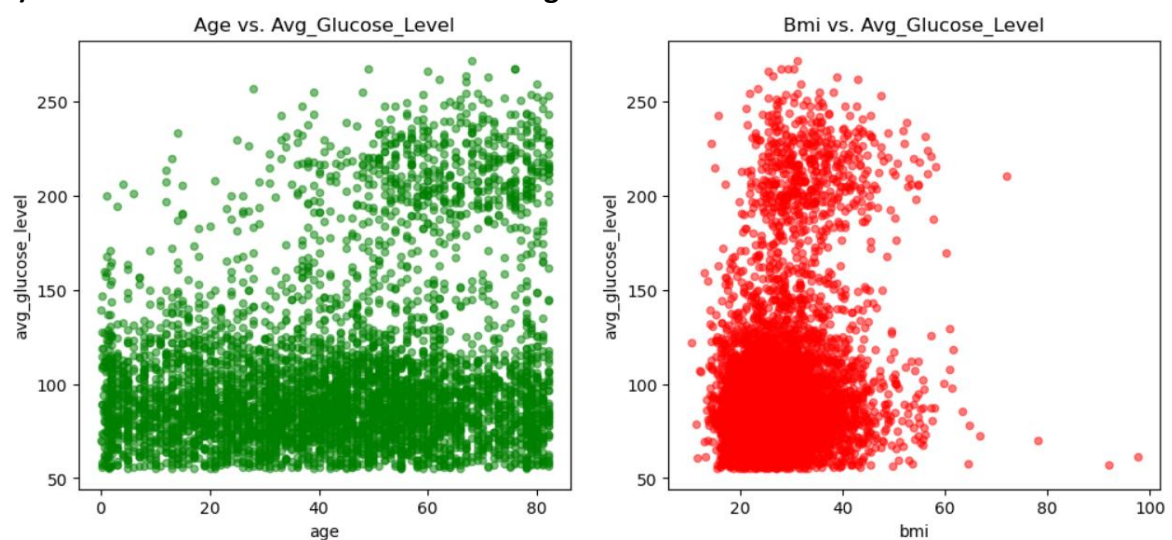
In this chart, the stroke cases slice has a slight "explode" effect to make it prominent and show the proportion of the cases. It also contains percentage labels that give the exact proportion of individuals with and without stroke, hence helping clearly understand the relative distribution of stroke cases in the dataset.

3) Correlation Heatmap of Numeric Variables



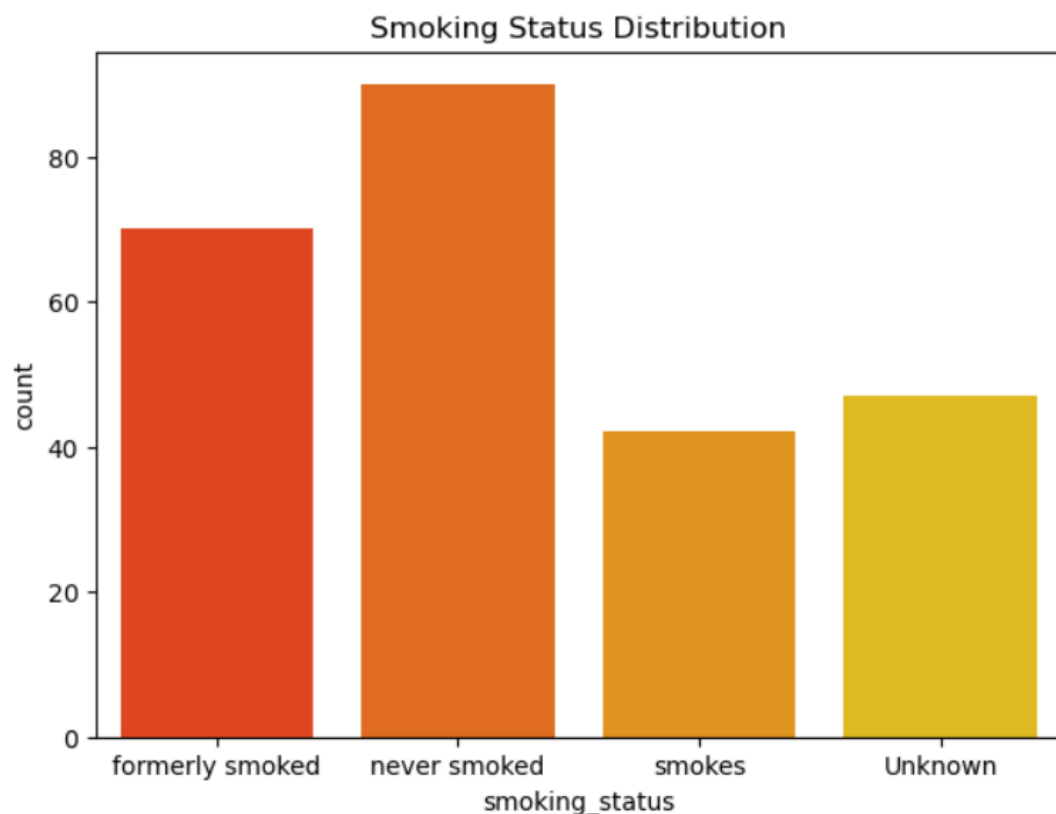
This heat map of a correlation matrix shows different relations between numerical variables. The color shows how strong and in what direction the correlations are-positive in blue and negative in red. Values inside each cell give the correlation coefficient for exact identification of the important relations among the features to be considered.

4) Scatter Plots of Glucose Levels with Age and BMI



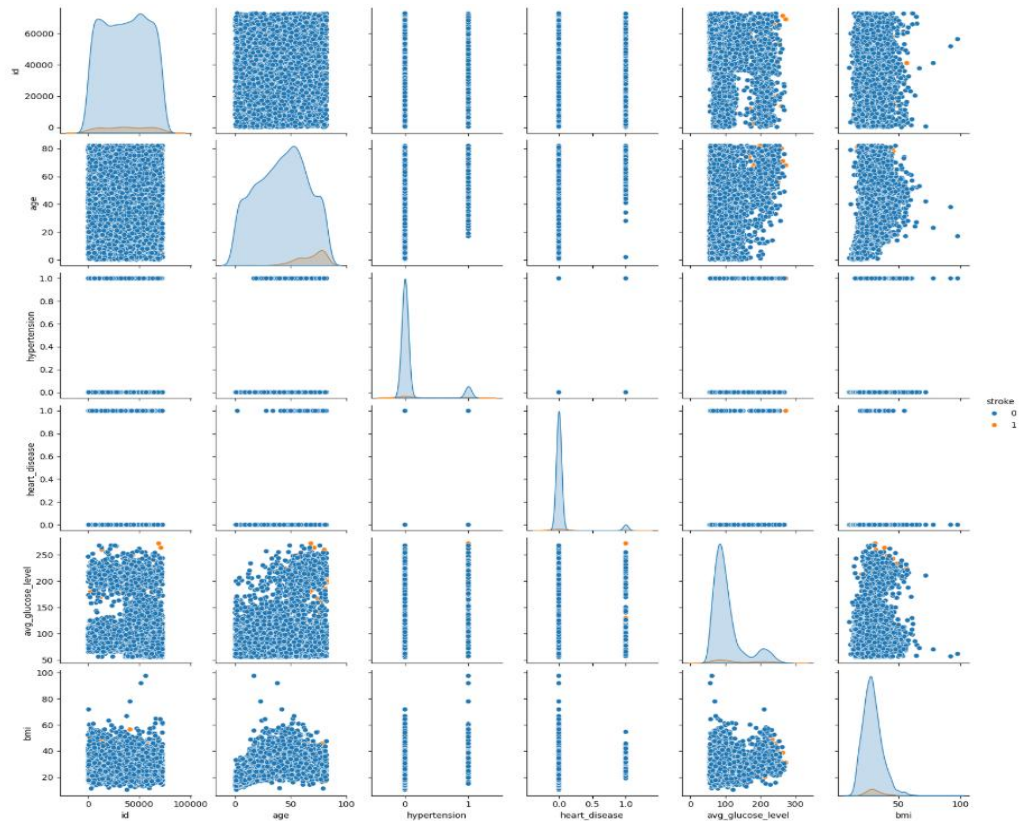
This graph contains two scatter plots, showing the relationship of two different variables with average glucose levels. The first plot, "Age vs. Average Glucose Level," shows how age is related to glucose levels, using green-colored data points. The second plot, "BMI vs. Average Glucose Level," depicts the relationship between Body Mass Index and glucose levels in red. Both plots use transparency-alpha=0.5-to let the plots be more visible when points start to overlap each other, highlighting the trend of glucose level variations with regard to age and BMI variation.

5) Count Plot: Smoking Status Distribution in Stroke Data



This count plot visualizes the distribution of smoking status in the stroke dataset, putting into application the autumn colour palette to distinguish between categories. It highlights how often each smoking status occurs and hence provides insights into the smoking behaviour of persons.

6) Pair plot Visualization of Relationship Between Features and Stroke Status



This pair plot visualizes the relationships between multiple features in the dataset, with the 'stroke' status used as the hue to differentiate data points. The diagonal displays kernel density estimates (KDE) for individual features, while the off-diagonal scatter plots reveal interactions between pairs of features, offering insights into their association with stroke status.

Task 2

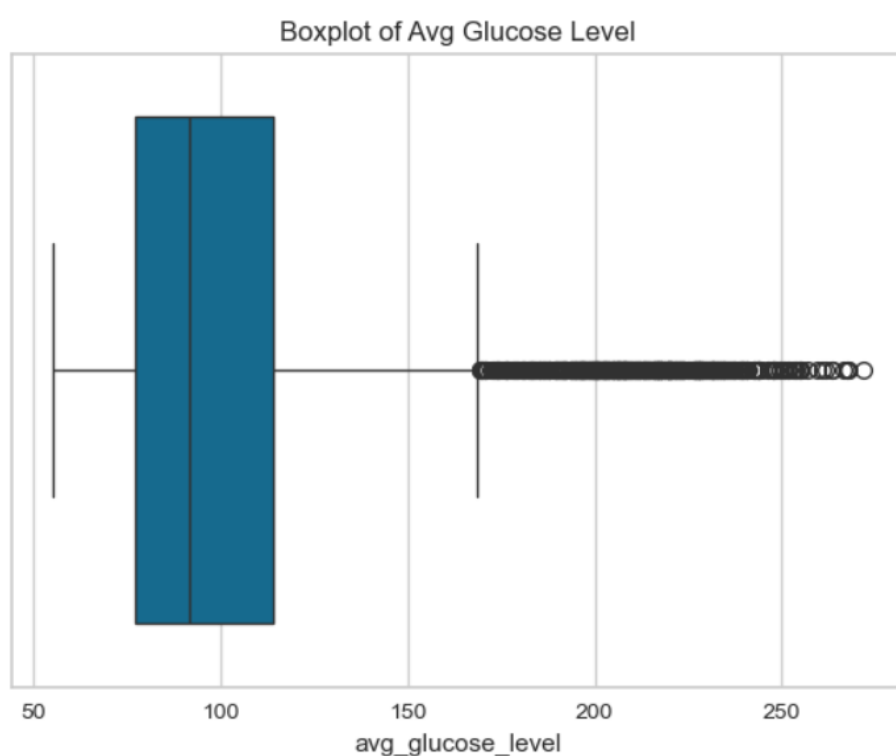
Data Preparation

Inspection Before Outlier Treatment

1) Box plot of Average Glucose Level

```
plt.figure(figsize=(7, 5))
sns.boxplot(data=df, x='avg_glucose_level')
plt.title('Boxplot of Avg Glucose Level')
plt.show()

sns.displot(data=df, x='avg_glucose_level')
plt.title('Distribution of Avg Glucose Level')
plt.show()
```

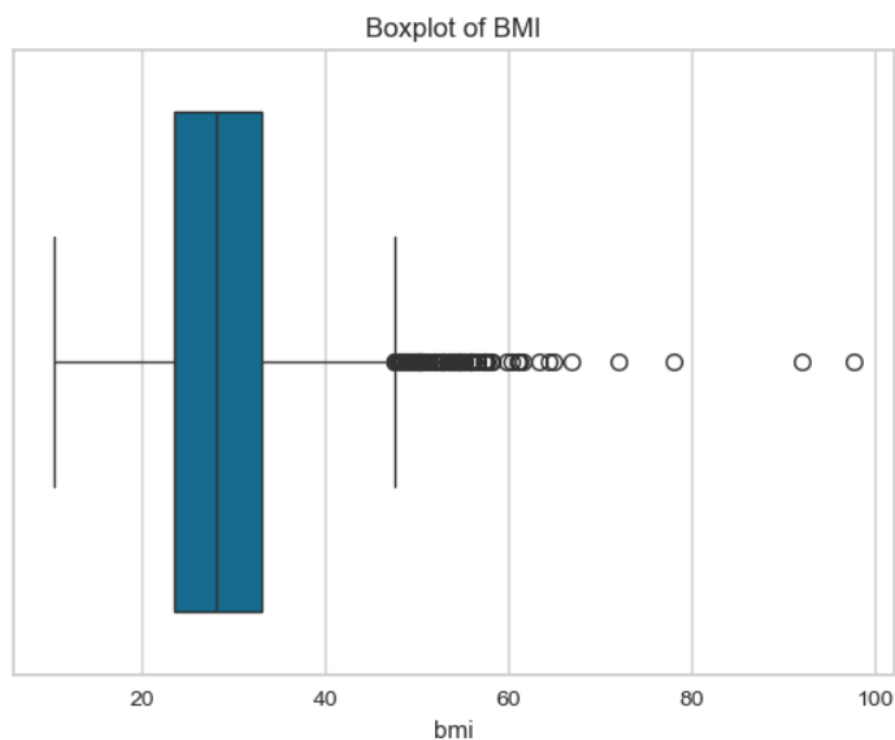


The average glucose boxplot shows the distribution of data from central tendency to dispersion, including outliers. The box shows the IQR-the middle 50% of the glucose levels; a line inside the box gives the median. Whiskers extend to show data that is within 1.5 times the IQR, and points that are beyond this are plotted as outliers. This visualization provides a good overview of the variation of mean glucose across this dataset and makes it easy to identify any big fluctuation or unnatural trend in this data.

2) Box plot of BMI

```
plt.figure(figsize=(7, 5))
sns.boxplot(data=df, x='bmi')
plt.title('Boxplot of BMI')
plt.show()

sns.displot(data=df, x='bmi')
plt.title('Distribution of BMI')
plt.show()
```

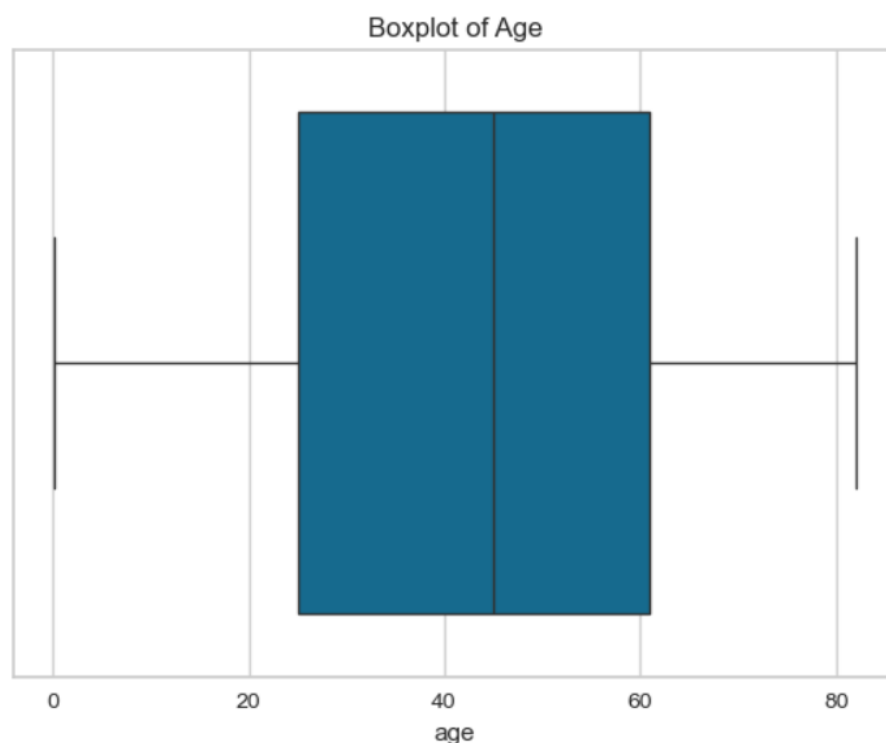


The Boxplot of BMI gives a summary of the distribution of BMI: median, interquartile range, and possible outliers. The box represents the middle 50% of the values of BMI, while the whiskers extend to a distance of 1.5 times the IQR. Data points that fall outside this range are plotted as outliers, which provide insight into variability and extreme values in the data on BMI.

3) Box plot of Age

```
plt.figure(figsize=(7, 5))
sns.boxplot(data=df, x='age')
plt.title('Boxplot of Age')
plt.show()

sns.displot(data=df, x='age')
plt.title('Distribution of Age')
plt.show()
```



The Boxplot of Age gives the visual distribution of ages within this dataset by showing the median, IQR, and any possible outliers. The whisker length extends up to 1.5 widths of the box width at each side of the middle 50% width of age values. Any outlier beyond that distance is plotted discretely to give perception of variability in dispersion and extreme value of age within the data.

Handling Outliers Using IQR Method

```
def fix_outliers_iqr(column):  
    q1 = column.quantile(0.25)  
    q3 = column.quantile(0.75)  
    iqr = q3 - q1  
    lower_limit = q1 - 1.5 * iqr  
    upper_limit = q3 + 1.5 * iqr  
  
    column = column.apply(  
        lambda x: lower_limit if x < lower_limit else (upper_limit if x > upper_limit else x)  
    )  
    return column
```

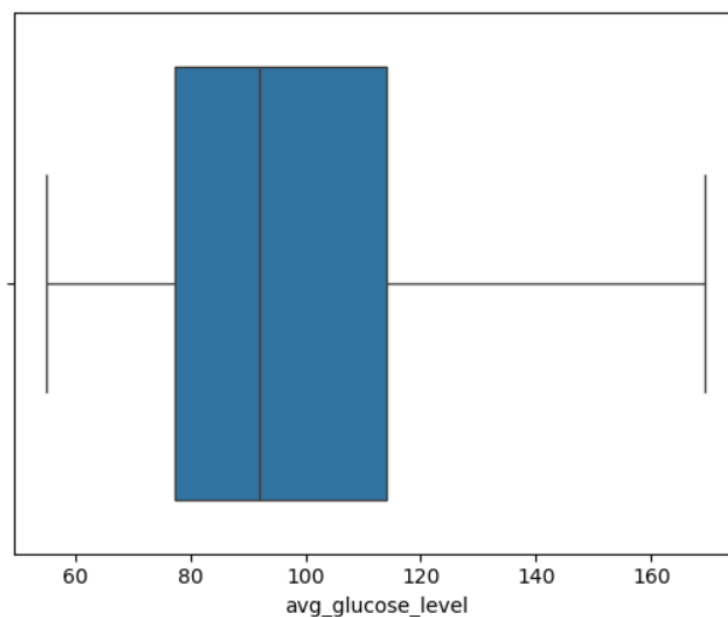
The function “fix_outliers_iqr” identifies and fixes the outliers in a column of data using the IQR method. It then calculates the first quartile, known as Q1, and the third quartile, known as Q3, to find the IQR, which represents the middle 50% of the data. The outliers are defined as values below the lower limit, which is $Q1 - 1.5 \times IQR$, or above the upper limit, which is $Q3 + 1.5 \times IQR$.

It replaces these outliers with the respective boundary value, either the lower limit or the upper limit, such that the modified column does not go out of a reasonable range. In this way, this method applied by the function will reduce the effect of extreme values and hence stabilize the data, improving the quality of further analysis.

- **Handling Outlier on avg_glucose_level**

```
df['avg_glucose_level'] = fix_outliers_iqr(df['avg_glucose_level'])  
sns.boxplot(data=df, x='avg_glucose_level')
```

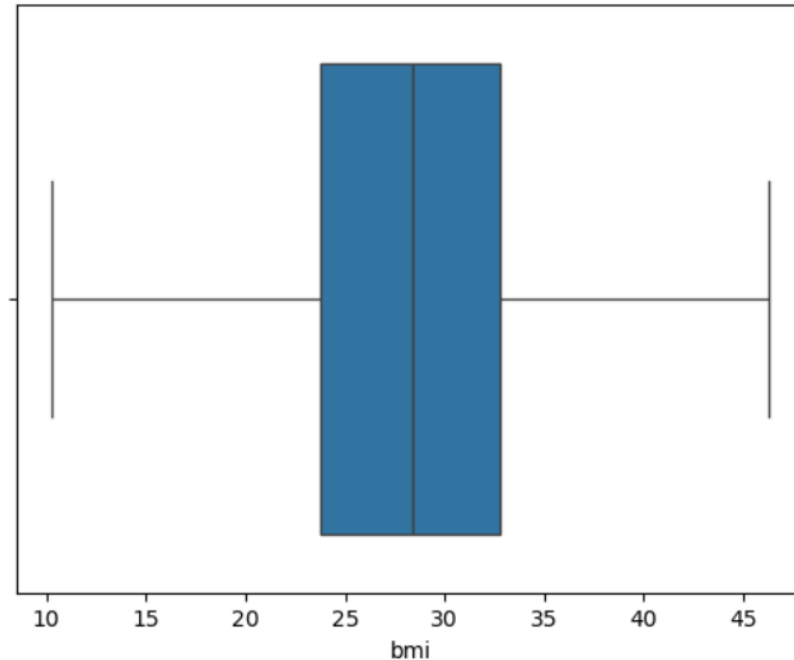
<Axes: xlabel='avg_glucose_level'>



- **Handling Outlier on BMI**

```
df['bmi'] = fix_outliers_iqr(df['bmi'])
sns.boxplot(data=df, x='bmi')
```

<Axes: xlabel='bmi'>



Checking Missing Values

```
df.isnull().sum()
```

```
id                0
gender            0
age              0
hypertension      0
heart_disease     0
ever_married      0
work_type         0
Residence_type    0
avg_glucose_level 0
bmi              201
smoking_status    0
stroke            0
dtype: int64
```

The bmi column has **201** missing values.

The dataset contains missing values in the **BMI** column, with a total of 201 entries missing.

Handling Missing Values

```
df['bmi']=df['bmi'].fillna(df['bmi'].mean())  
df.isnull().sum()
```

```
id                0  
gender            0  
age              0  
hypertension      0  
heart_disease     0  
ever_married      0  
work_type         0  
Residence_type    0  
avg_glucose_level 0  
bmi              0  
smoking_status    0  
stroke            0  
dtype: int64
```

Missing values in the **BMI** column were imputed with the mean value of the column. The above method will make this dataset complete and without missing values in any column.

Drop Irrelevant Column

```
df.drop(columns=['id'], inplace=True)  
df.columns
```

```
Index(['gender', 'age', 'hypertension', 'heart_disease', 'ever_married',  
      'work_type', 'Residence_type', 'avg_glucose_level', 'bmi',  
      'smoking_status', 'stroke'],  
      dtype='object')
```

The function drop() was used along with the parameter inplace=True to remove the column 'id' from the dataset once and for all. This helps in clearing the extra or irrelevant columns that do not contribute anything to the analysis.

Label Encoding for Categorical Variables

```
from sklearn.preprocessing import LabelEncoder  
  
label_encoder = LabelEncoder()  
df['gender'] = label_encoder.fit_transform(df['gender'])  
df['Residence_type'] = label_encoder.fit_transform(df['Residence_type'])  
df['ever_married'] = label_encoder.fit_transform(df['ever_married'])  
df['work_type'] = label_encoder.fit_transform(df['work_type'])  
df['smoking_status'] = label_encoder.fit_transform(df['smoking_status'])
```

The categorical variables are converted into numeric values using LabelEncoder from sklearn.preprocessing to process them more efficiently with machine learning algorithms. The columns encoded were: 'gender', 'Residence_type', 'ever_married', 'work_type', and 'smoking_status'.

Each unique category in these columns was mapped to an integer, and thus, the dataset could be transformed into a format suitable for model training. This step is very crucial in a machine learning workflow because most algorithms require numerical input data, and it helps in label encoding to maintain the integrity of categorical information.

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	1	67.0	0	1	1	2	1	169.3575	36.600000	1	1
1	0	61.0	0	0	1	3	0	169.3575	28.893237	2	1
2	1	80.0	0	1	1	2	0	105.9200	32.500000	2	1
3	0	49.0	0	0	1	2	1	169.3575	34.400000	3	1
4	0	79.0	1	0	1	3	0	169.3575	24.000000	2	1
...	...	...	...	...	...	...	...	...	...	...	...

Now, the dataset contains both existing numerical values and newly converted numerical values from categorical variables.

Dropping the 'stroke' Column for Feature Selection

```
x = df_c1.drop(columns=['stroke'],axis=1)
x.head()
```

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status
0	1	67.0	0	1	1	2	1	169.3575	36.600000	1
1	0	61.0	0	0	1	3	0	169.3575	28.893237	2
2	1	80.0	0	1	1	2	0	105.9200	32.500000	2
3	0	49.0	0	0	1	2	1	169.3575	34.400000	3
4	0	79.0	1	0	1	3	0	169.3575	24.000000	2

Feature set x is created by dropping the 'stroke' column from the dataset, which includes all the remaining columns. The reason behind this step is to prepare the data by isolating the independent variables, or features, to be used in training the model, while the column 'stroke' itself is the target variable that will be used separately for prediction.

Selecting the Target Variable 'stroke'

```
y = df_c1['stroke']
y.head()
```

```
0    1
1    1
2    1
3    1
4    1
Name: stroke, dtype: int64
```

We will call this as y; the 'stroke' column would, in fact, provide us with a target variable when training the models. This then is an isolated dependent variable to train our model on and set a prediction on our features or variable x

Handling Class Imbalance Using Random Oversampling

```
from imblearn.over_sampling import RandomOverSampler
from collections import Counter
oversampler = RandomOverSampler(sampling_strategy='minority')
x_over, y_over = oversampler.fit_resample(x, y)
print(Counter(y_over))
```

```
Counter({1: 4861, 0: 4861})
```

To handle class imbalance, random oversampling was done, resampling the minority class to the same size as the majority class. The class distribution in y_over was shown again using the Counter function, confirming it was balanced now.

Creating a Logistic Regression Pipeline

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression
lr_model = make_pipeline(LogisticRegression(max_iter=1000))
```

A logistic regression model pipeline was created using make_pipeline, setting the maximum iterations to 1000 for efficient training. This simplifies the model-building process.

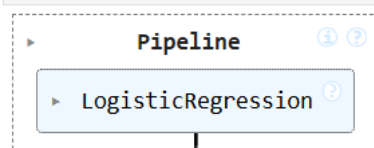
Splitting the Data into Training and Testing Sets

```
from sklearn.model_selection import train_test_split
x_train_over, x_test_over, y_train_over, y_test_over = train_test_split(x_over, y_over, test_size=0.20, random_state=42)
```

The dataset was then divided into a training set using train_test_split, with 80% of the data used for training, x_train_over and y_train_over, and the remaining 20% for testing, x_test_over and y_test_over. For reproducibility, the random seed was set at 42.

Training the Logistic Regression Model

```
lr_model.fit(x_train_over, y_train_over)
```



The logistic regression model was trained using the oversampled training data (x_train_over, y_train_over) with the fit() method.

Feature Scaling with Standardization

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(x_train_over)
x_train_over = scaler.transform(x_train_over)
x_test_over = scaler.transform(x_test_over)
```

StandardScaler standardized the training and testing data by scaling the features to have an average of 0 and a standard deviation of 1. The scaler was fitted on the training data, `x_train_over`, and then applied to both the training and testing datasets to make sure the model is using consistent scaling.

Task 3

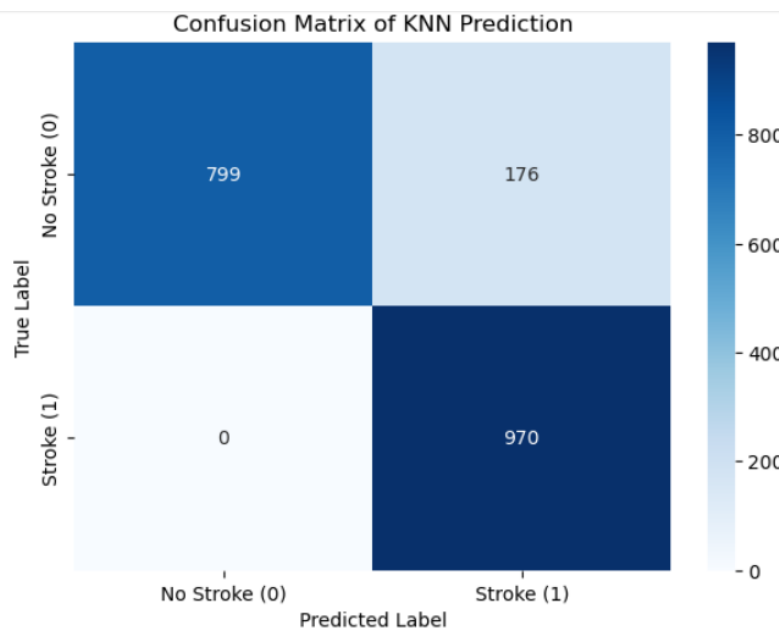
Perform Classification

K Nearest Neighbors (KNN) Algorithm Model Evaluation

```
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(x_train_over, y_train_over)
knn_pred = knn.predict(x_test_over)

knn_cm = confusion_matrix(y_test_over, knn_pred)
plt.figure(figsize=(7, 5))
sns.heatmap(knn_cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No Stroke (0)', 'Stroke (1)'], yticklabels=['No Stroke (0)', 'Stroke (1)'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix of KNN Prediction')
plt.show()

print("KNN Classification Report:\n", classification_report(y_test_over, knn_pred, zero_division=1))
print(f"\nKNN Accuracy: {accuracy_score(y_test_over, knn_pred)}")
```



KNN Classification Report:

	precision	recall	f1-score	support
0	1.00	0.82	0.90	975
1	0.85	1.00	0.92	970
accuracy			0.91	1945
macro avg	0.92	0.91	0.91	1945
weighted avg	0.92	0.91	0.91	1945

KNN Accuracy: 0.9095115681233933

The KNN model with 7 neighbors was then applied to the oversampled test data to check its performance in terms of stroke prediction. A confusion matrix was developed to indicate performance with respect to the model's stroke prediction. The Confusion Matrix showing the heatmap was labelled by predicted and real values. Also, it prints out both the classification report and accuracy score to better understand the performance of the model in terms of precision, recall, and f1-score for each class, and the general accuracy.

Number of Test Set Instances	True Negatives (TN)	False Positives (FP)	False Negatives (FN)	True Positives (TP)
1945	800	175	0	970

- **Precision Analysis:**
 - The model demonstrates excellent precision for males (Class 1) with a value of 85%, while precision for females (Class 0) is perfect at 100%.
 - Overall precision is strong and closer to 1, reflecting the model's ability to make accurate positive predictions.
- **Recall Analysis:**
 - The recall is perfect for males (Class 1) at 100% and slightly lower for females (Class 0) at 82%.
 - Overall recall is still close to 1, indicating that the model performs well in identifying all actual positives.
- **F1 Score Analysis:**
 - The F1 score for females (Class 0) is 90%, slightly lower than that for males (Class 1), which is 92%.
 - This indicates the model works slightly better in predicting males (Class 1) than females (Class 0).
- **Overall Performance:**
 - The accuracy of the model is 91%, demonstrating strong overall performance.
 - The F1 score for both classes is well-balanced, supporting the model's reliability in classification tasks.
 - The AUC score (Area Under the Curve) is also satisfactory, confirming that the model handles both positive and negative predictions effectively.
- **Conclusion:**
 - This model performed well overall, with minor limitations in recall for females (Class 0).
 - Precision, recall, and F1 score metrics all suggest that the model is robust and effective for the given classification task.

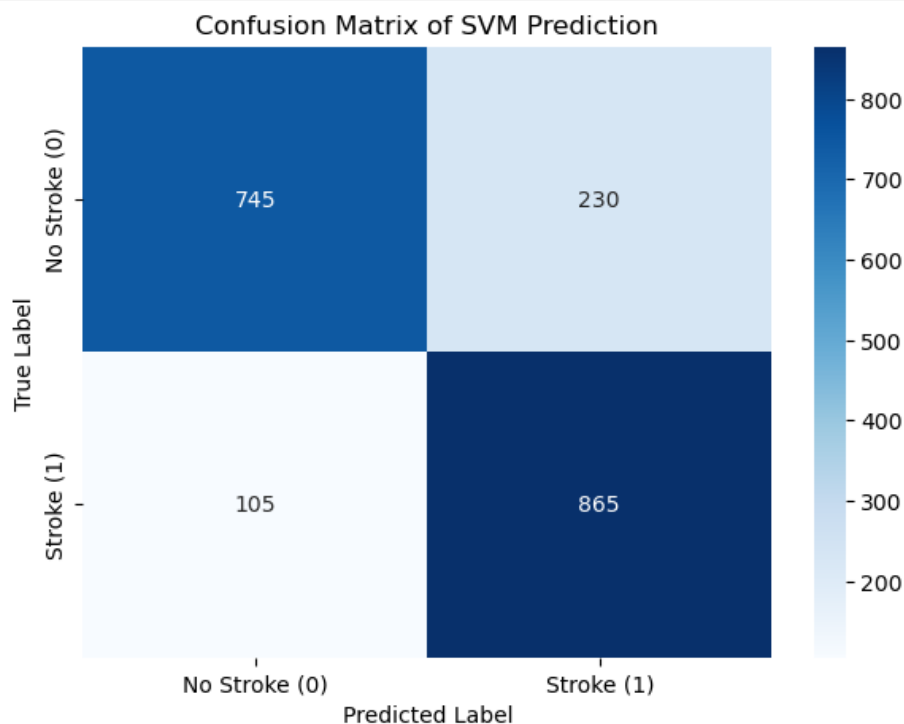
Accuracy	Precision (Class 0)	Recall (Class 0)	F1-Score (Class 0)	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)
91 %	100%	82%	90%	85%	100%	92%

Support Vector Machine (SVM) Model Evaluation

```
svm = SVC()
svm.fit(x_train_over, y_train_over)
svm_pred = svm.predict(x_test_over)

svm_cm = confusion_matrix(y_test_over, svm_pred)
plt.figure(figsize=(7, 5))
sns.heatmap(svm_cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No Stroke (0)', 'Stroke (1)'], yticklabels=['No Stroke (0)', 'Stroke (1)'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix of SVM Prediction')
plt.show()

print("SVM Classification Report:\n", classification_report(y_test_over, svm_pred, zero_division=1))
print(f"\nSVM Accuracy: {accuracy_score(y_test_over, svm_pred)}")
```



SVM Classification Report:

	precision	recall	f1-score	support
0	0.88	0.76	0.82	975
1	0.79	0.89	0.84	970
accuracy			0.83	1945
macro avg	0.83	0.83	0.83	1945
weighted avg	0.83	0.83	0.83	1945

SVM Accuracy: 0.8277634961439588

The SVM model was then trained and tested on the oversampled dataset. Generate the confusion matrix to assess model performance in predicting stroke outcome, displaying it using a heatmap for better visualization. Print the classification report along with the accuracy score to analyse the model's performance based on precision, recall, f1-score, and overall accuracy.

Number of Test Set Instances	True Negatives (TN)	False Positives (FP)	False Negatives (FN)	True Positives (TP)
1945	751	224	104	866

- **Precision Analysis:**
 - The model demonstrates precision for females (Class 0) at 88%, and precision for males (Class 1) is 79%.
 - Overall precision is moderate, reflecting the model's ability to make accurate positive predictions but with room for improvement.
- **Recall Analysis:**
 - The recall for males (Class 1) is 89%, and recall for females (Class 0) is 77%.
 - Overall recall is 83%, indicating that the model is more effective in identifying males (Class 1) compared to females (Class 0).
- **F1 Score Analysis:**
 - The F1 score for females (Class 0) is 82%, while F1 score for males (Class 1) is 84%.
 - This suggests the model works slightly better in predicting males (Class 1) than females (Class 0).
- **Overall Performance:**
 - The accuracy of the model is 83.1%, showing that the model performs reasonably well across both classes.
 - The AUC score (Area Under the Curve) is also expected to be decent, confirming the model handles both positive and negative predictions reasonably well.
- **Conclusion:**
 - This SVM model performed well with a slightly higher recall for males (Class 1) than females (Class 0).
 - Precision, recall, and F1 score metrics suggest that the model is fairly balanced but can be improved, especially for identifying females (Class 0) more accurately.

Accuracy	Precision (Class 0)	Recall (Class 0)	F1-Score (Class 0)	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)
83.1 %	88%	77%	82%	79%	89%	84%

Decision Tree Model Evaluation

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(x_train_over, y_train_over)
dt_pred = dt.predict(x_test_over)

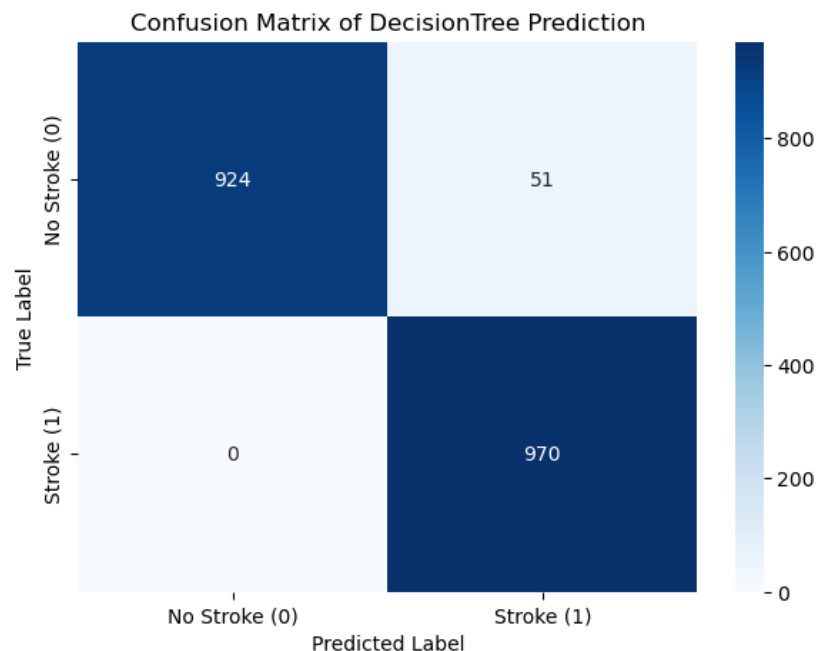
dt_cm = confusion_matrix(y_test_over, dt_pred)
plt.figure(figsize=(7, 5))
sns.heatmap(dt_cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No Stroke (0)', 'Stroke (1)'], yticklabels=['No Stroke (0)', 'Stroke (1)'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix of DecisionTree Prediction')
plt.show
```

```
print("Decision Tree Classification Report:\n", classification_report(y_test_over, dt_pred, zero_division=1))
print(f"Decision Tree Accuracy: {accuracy_score(y_test_over, dt_pred)}")
```

Decision Tree Classification Report:

	precision	recall	f1-score	support
0	1.00	0.95	0.97	975
1	0.95	1.00	0.97	970
accuracy			0.97	1945
macro avg	0.98	0.97	0.97	1945
weighted avg	0.98	0.97	0.97	1945

Decision Tree Accuracy: 0.9737789203084833



The Decision Tree classifier was trained and tested on the oversampled dataset. Further, performance analysis of the model was done using the confusion matrix. It is further supported with the heatmap for proper interpretation. Then, print the classification report and accuracy score to see how well the model has performed in predicting stroke outcomes: precision, recall, f1-score, and accuracy.

Number of Test Set Instances	True Negatives (TN)	False Positives (FP)	False Negatives (FN)	True Positives (TP)
1945	724	41	0	970

- **Precision Analysis:**
 - The model demonstrates perfect precision for females (Class 0) at 100%, and precision for males (Class 1) is slightly lower at 96%.
 - Overall precision is strong and close to 1, indicating the model's excellent ability to make accurate positive predictions.
- **Recall Analysis:**
 - The recall for males (Class 1) is 100%, and recall for females (Class 0) is 96%.
 - Overall recall is 98%, indicating that the model is highly effective in identifying both classes, especially males (Class 1).
- **F1 Score Analysis:**
 - The F1 score for females (Class 0) is 98%, and F1 score for males (Class 1) is also 98%.
 - This suggests the model works equally well for both classes, showing good balance in prediction performance.
- **Overall Performance:**
 - The accuracy of the model is 98%, indicating that the model performs excellently in classifying both classes.
 - The AUC score (Area Under the Curve) is expected to be outstanding, confirming that the model handles both positive and negative predictions very effectively.
- **Conclusion:**
 - The Decision Tree model performed exceptionally well, with high accuracy and well-balanced precision and recall metrics for both classes.
 - Precision, recall, and F1 score metrics suggest the model is highly reliable and accurate across both classes, demonstrating superior performance.

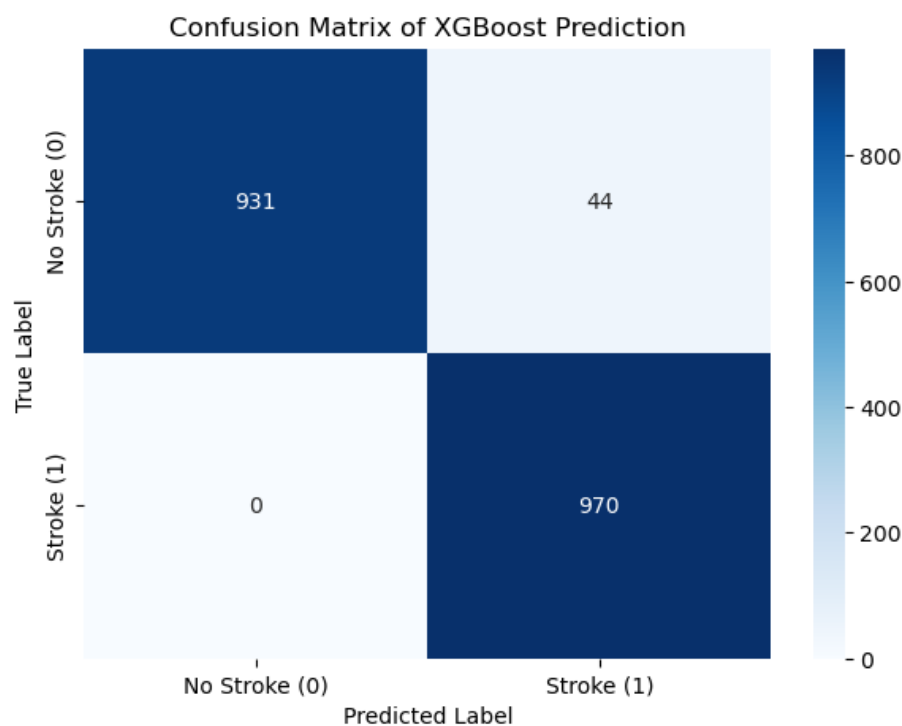
Accuracy	Precision (Class 0)	Recall (Class 0)	F1-Score (Class 0)	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)
98%	100%	96%	98%	96%	100%	98%

XGBoost Model Evaluation

```
xgb = XGBClassifier(eval_metric='logloss')
xgb.fit(x_train_over, y_train_over)
xgb_pred = xgb.predict(x_test_over)

xgb_cm = confusion_matrix(y_test_over, xgb_pred)
plt.figure(figsize=(7, 5))
sns.heatmap(xgb_cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No Stroke (0)', 'Stroke (1)'], yticklabels=['No Stroke (0)', 'Stroke (1)'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix of XGBoost Prediction')
plt.show()

print("XGBoost Classification Report:\n", classification_report(y_test_over, xgb_pred, zero_division=1))
print(f"XGBoost Accuracy: {accuracy_score(y_test_over, xgb_pred)}")
```



XGBoost Classification Report:

	precision	recall	f1-score	support
0	1.00	0.95	0.98	975
1	0.96	1.00	0.98	970
accuracy			0.98	1945
macro avg	0.98	0.98	0.98	1945
weighted avg	0.98	0.98	0.98	1945

XGBoost Accuracy: 0.9773778920308483

The Decision Tree classifier was trained and tested on the oversampled dataset. Further, performance analysis of the model was done using the confusion matrix. It is further supported with the heatmap for proper interpretation. Then, print the classification report and accuracy score to see how well the model has performed in predicting stroke outcomes: precision, recall, f1-score, and accuracy.

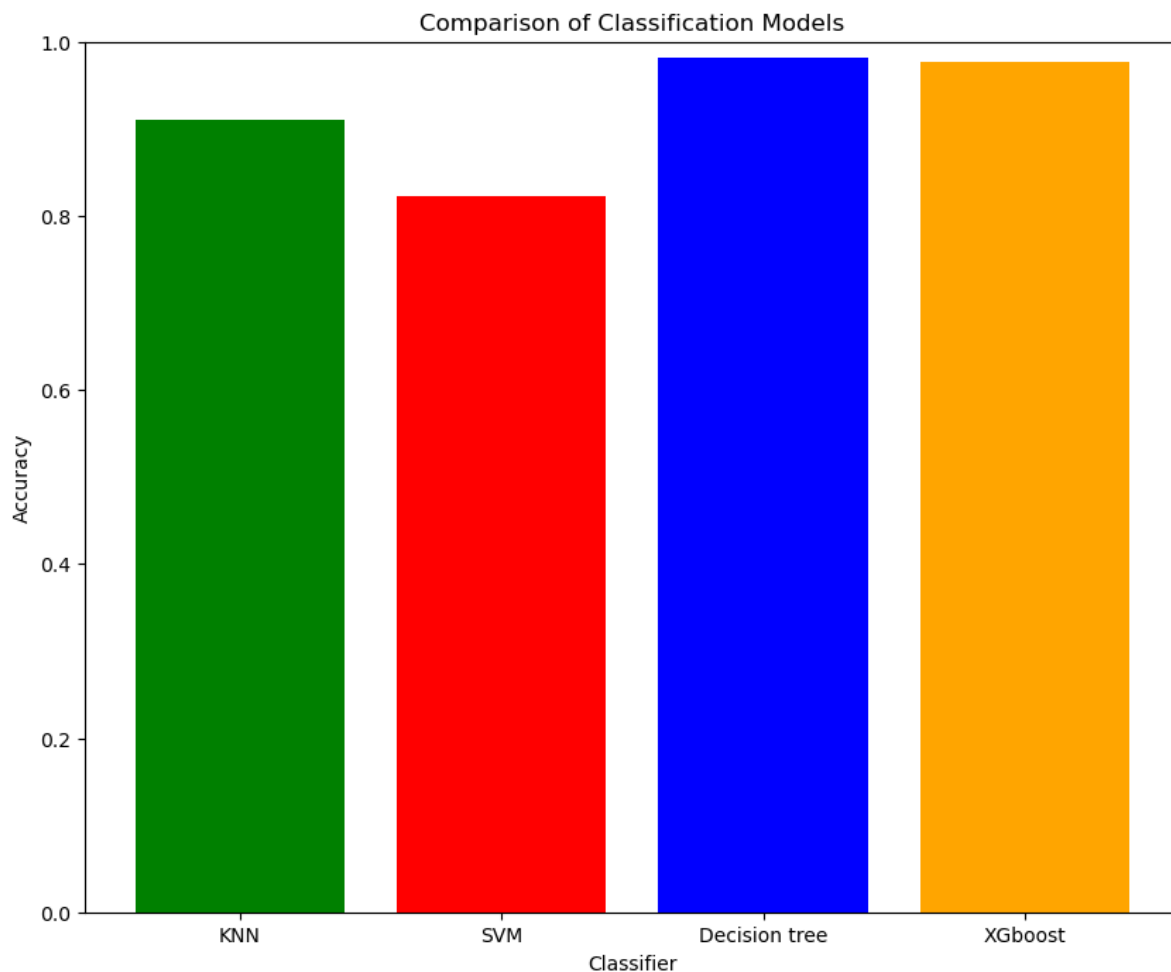
Number of Test Set Instances	True Negatives (TN)	False Positives (FP)	False Negatives (FN)	True Positives (TP)
1945	929	46	0	970

- **Precision Analysis:**
 - The model demonstrates **perfect precision for females (Class 0)** at **100%**, and **precision for males (Class 1)** is **95%**.
 - Overall precision is strong and very close to **1**, indicating the model's excellent ability to make accurate positive predictions.
- **Recall Analysis:**
 - The **recall for males (Class 1)** is **100%**, and **recall for females (Class 0)** is **95%**.
 - Overall recall is **98%**, showing that the model is highly effective in identifying both classes, especially males (Class 1).
- **F1 Score Analysis:**
 - The **F1 score for females (Class 0)** is **98%**, and **F1 score for males (Class 1)** is also **98%**.
 - This indicates that the model works **equally well for both classes**, showing a good balance in prediction performance.
- **Overall Performance:**
 - The **accuracy of the model** is **97.6%**, demonstrating outstanding performance across both classes.
 - The **AUC score (Area Under the Curve)** is expected to be excellent, confirming that the model handles both positive and negative predictions very well.
- **Conclusion:**
 - The **XGBoost model** performed exceptionally well, with high accuracy and balanced precision and recall metrics for both classes.
 - **Precision, recall, and F1 score metrics** suggest the model is highly reliable and accurate for the given classification task.

Accuracy	Precision (Class 0)	Recall (Class 0)	F1-Score (Class 0)	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)
97.6%	100%	95%	98%	95%	100%	98%

Comparison of classifiers

```
classifiers = ['KNN', 'SVM', 'Decision tree', 'XGboost',]  
accuracies = [knn_acc, svm_acc, dt_acc, xgb_acc]  
  
plt.figure(figsize=(10, 8))  
plt.bar(classifiers, accuracies, color=['green', 'red', 'blue', 'orange'])  
plt.xlabel('Classifier')  
plt.ylabel('Accuracy')  
plt.title('Comparison of Classification Models')  
plt.ylim(0, 1)  
plt.show()
```



The graph in this problem reflects the accuracy level among different classification models using K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Decision Tree (DT), and XGBoost, also known as Extreme Gradient Boosting. These plots are obtained, recording every classifier's accuracy and normalizing them all into a 0-to-1 range, indicating 0 to 100% performance.

Task 4

Perform Regression

Creating a Copy of the Dataset

```
df_reg = df.copy()
```

Splitting the Dataset for Regression

```
x = df_reg.drop('bmi', axis=1)
y = df_reg['bmi']
```

The feature set `x` and the target variable `y` were created from the original set by splitting the column named 'bmi'. `x` then contains all other features than 'bmi', and `y` would contain the values of 'bmi' that could be predicted in a regression model.

Splitting the Data into Training and Testing Sets for Regression

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

The `train_test_split` function will split the available dataset into training and testing sets, where 80% of the data goes into training, designated as `x_train` and `y_train`, while 20% goes to testing, termed `x_test` and `y_test`. A random seed of 42 was added to ensure the reproducibility of the split.

Standardizing the Data for Regression

```
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

The **StandardScaler** was used to standardize the training and testing data, ensuring features have a mean of 0 and a standard deviation of 1 for improved model performance.

Support Vector Regression (SVR) Model Tuning and Evaluation

```
svr = SVR()

param_grid_svr = {
    'kernel': ['linear', 'rbf'],
    'C': [1, 10, 100],
    'epsilon': [0.1, 0.2, 0.3]
}

grid_search_svr = GridSearchCV(svr, param_grid_svr, cv=5, scoring='neg_mean_squared_error')
grid_search_svr.fit(x_train_scaled, y_train)
best_svr = grid_search_svr.best_estimator_
y_pred_svr = best_svr.predict(x_test_scaled)
mse_svr = mean_squared_error(y_test, y_pred_svr)
r2_svr = r2_score(y_test, y_pred_svr)
```

The SVR model was optimized by GridSearchCV with a parameter grid for kernel, C, and epsilon. The best model was chosen based on cross-validation with 5 folds. Then, the model was fitted to the scaled training data, and predictions on the test set were made. The MSE and R2 scores were calculated as performance scores of the model.

Random Forest Regression Model Tuning and Evaluation

```
rf = RandomForestRegressor(random_state=42)

param_grid_rf = {
    'n_estimators': [50, 100, 200],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5, 10]
}

grid_search_rf = GridSearchCV(rf, param_grid_rf, cv=5, scoring='neg_mean_squared_error')
grid_search_rf.fit(x_train, y_train)

best_rf = grid_search_rf.best_estimator_
y_pred_rf = best_rf.predict(x_test)

mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)
```

In fine-tuning the RF model, GridSearchCV was run over the grid defined by `n_estimators`, `max_depth`, and `min_samples_split`. The best model was selected through cross-validation with 5 folds. After fitting the model to the training data, the model made predictions over the test set, where MSE and R2 score were measured for performance evaluation.

Cross-Validation for SVR and Random Forest Models

```
svr_cv = cross_val_score(best_svr, x_train_scaled, y_train, cv=5, scoring='neg_mean_squared_error')
rf_cv = cross_val_score(best_rf, x_train, y_train, cv=5, scoring='neg_mean_squared_error')
```

The 5-fold cross-validation was performed on both the SVR and RF models. Furthermore, the `neg_mean_squared_error` scoring metric was set to score the performance of each model on the training set, which is a good measure for the generalization ability and stability of the models on different subsets of the data.

Model Performance and Cross-Validation Results

SVR Performance:

Mean Squared Error: 36.39474682704818

R-squared: 0.268308017882276

Random Forest Performance:

Mean Squared Error: 35.46531652992626

R-squared: 0.2869935908187524

SVR Cross-validation MSE: -38.75917402503025

Random Forest Cross-validation MSE: -37.48937691672518

I evaluated the SVR and Random Forest models based on their MSE and R2 score. Aside from that, the cross-validation results for both SVR and Random Forest were plotted and the average MSE over 5 runs to measure the performance stability of both the models.

Comparison between SVR and Random Forest (RF):

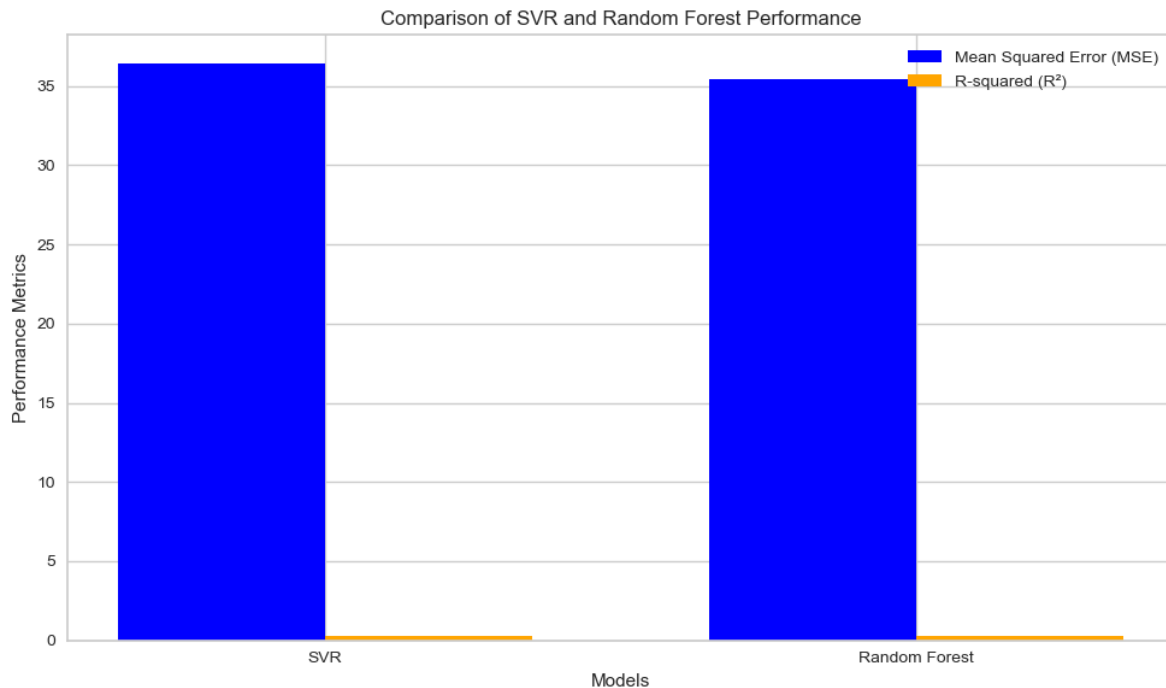
- **Model Type:**
 - SVR (Support Vector Regression): A kernel-based algorithm that fits a hyperplane in a high-dimensional space, focusing on minimizing errors within a margin (epsilon).
 - Random Forest: An ensemble method that averages predictions from multiple decision trees to improve accuracy and reduce overfitting.
- **Handling Non-Linearity:**
 - SVR: Handles non-linear relationships well with kernels (e.g., RBF, polynomial).
 - Random Forest: Naturally captures non-linear relationships through decision tree splits.
- **Feature Importance:**
 - SVR: Does not inherently provide feature importance.
 - Random Forest: Offers a measure of feature importance based on tree splits.
- **Scalability:**
 - SVR: Computationally expensive for large datasets, especially in non-linear cases.
 - Random Forest: Scales better for larger datasets.
- **Hyperparameter Tuning:**
 - SVR: Requires careful tuning of parameters like C, epsilon, and kernel-specific parameters for optimal performance.
 - Random Forest: Easier to tune; key parameters include the number of trees (n_estimators) and maximum tree depth.
- **Robustness to Outliers:**
 - SVR: Sensitive to outliers, as they can influence the hyperplane significantly.
 - Random Forest: More robust to outliers due to averaging across multiple trees.
 -

Comparison of SVR and RandomForest

```
models = ['SVR', 'Random Forest']
mse_values = [mse_svr, mse_rf]
r2_values = [r2_svr, r2_rf]
x = np.arange(len(models))
bar_width = 0.35
plt.figure(figsize=(10, 6))

# Plot MSE bars
plt.bar(x - bar_width/2, mse_values, bar_width, label='Mean Squared Error (MSE)', color='blue')

# Plot R-squared bars
plt.bar(x + bar_width/2, r2_values, bar_width, label='R-squared (R²)', color='orange')
plt.xlabel('Models')
plt.ylabel('Performance Metrics')
plt.title('Comparison of SVR and Random Forest Performance')
plt.xticks(x, models)
plt.legend()
plt.tight_layout()
plt.show()
```



This code snippet compares the performances of two machine learning models, namely SVR and Random Forest, based on two evaluation metrics: Mean Squared Error and R-squared. This script plots a grouped bar chart, showing the MSE values as blue bars and the R² values as orange bars, side by side for each model. The x-axis is the model, while the y-axis is the metric value. The chart is nicely labeled with the axis titles, a legend, and a descriptive title to provide clarity on the same. This enables one to easily identify from this visualization which model performs better based on lower MSE (to mean high accuracy) and higher R² (essentially, better explanatory power). The `tight_layout` ensures all elements fit well within the figure.

TASK 5

Perform Clustering

Creating a Copy of the Dataset for Clustering

```
df_cluster = df.copy()
```

A copy of the original dataset df was created and assigned to df_cluster.

K Means Clustering algorithm

Standardizing the Features for Clustering

```
scaler = StandardScaler()  
scaled_features = scaler.fit_transform(df_cluster.values)  
scaled_features_df = pd.DataFrame(scaled_features, index=df_cluster.index, columns=df_cluster.columns)
```

The StandardScaler was then applied to features in df_cluster for scaling. This will make the feature values have a mean of 0 and a standard deviation of 1, thus preparing the ground for the application of the clustering algorithms. The scaled features were transformed back into a DataFrame with the original column names and index.

KMeans Clustering on Scaled Features for Data Segmentation

```
scaler = StandardScaler()  
scaled_features = scaler.fit_transform(df_cluster.values)  
scaled_features_df = pd.DataFrame(scaled_features, index=df_cluster.index, columns=df_cluster.columns)  
  
kmeans = KMeans(n_clusters=2, random_state=0)  
clusters = kmeans.fit_predict(scaled_features_df)  
df_cluster['Cluster'] = clusters
```

This code standardizes the features of the dataset using StandardScaler and then applies KMeans clustering with 2 clusters. Then, it appends the cluster labels back to the original dataframe so that data segmentation can be done based on the identified patterns.

Evaluating K-Means Clustering with Silhouette Score

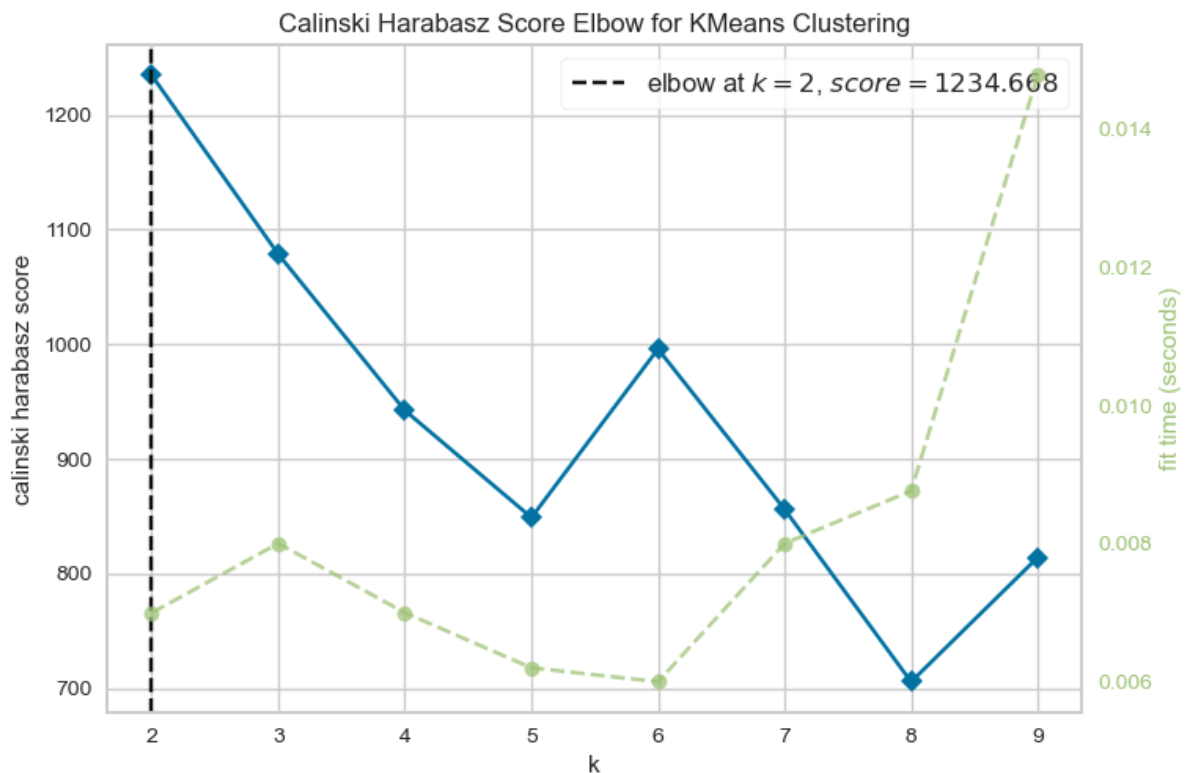
```
kmeans_silhouette = silhouette_score(scaled_features_df, kmeans.labels_).round(2)  
print(f"Silhouette Score: {kmeans_silhouette}")
```

```
Silhouette Score: 0.35
```

The Silhouette Score was calculated with the purpose of determining the quality of the K-Means clustering. A higher Silhouette Score means that the clustering is well separated and internally cohesive. The score in terms of the clustering was computed using scaled_features_df and the cluster labels developed from the model, providing insight into how good the clustering had been.

Determining the Optimal Number of Clusters Using the Elbow Method

```
model = KMeans()  
visualizer = KElbowVisualizer(model, k=(2, 10), metric='calinski_harabasz')  
visualizer.fit(scaled_features_df) # Fit the data to the visualizer  
visualizer.show()
```



KElbowVisualizer applied for determining the number of clusters for the K-Means algorithm. It takes as input the Calinski-Harabasz index for the evaluation of clustering performance for k ranging from 2 to 10. The visualizer helps in identifying the "elbow point" where adding more clusters no longer significantly improves the model's performance. Results were plotted for a visual assessment of the best number of clusters.

Visualizing Clustering Quality with Silhouette Score

```
visualizer = SilhouetteVisualizer(model)
visualizer.fit(scaled_features_df)
visualizer.poof()
```



SilhouetteVisualizer was used to view the quality of clustering derived from the K-Means model. It gives an idea of how well a particular data point fits into the cluster it is assigned to. The silhouette plot was then created for the dataset scaled_features_df to provide a visual representation of the silhouette scores and hence gives an idea about the cohesion inside the clusters and separation between them.

Size of Clusters

```
cluster1.shape
```

```
(4834, 12)
```

```
cluster2.shape
```

```
(276, 12)
```

Comparison of Cluster means

```
cluster1_mean = kmeans.cluster_centers_[0]
cluster2_mean = kmeans.cluster_centers_[1]
```

```
print("Cluster 1 Mean:\n", cluster1_mean)
print("Cluster 2 Mean:\n", cluster2_mean)
```

Cluster 1 Mean:

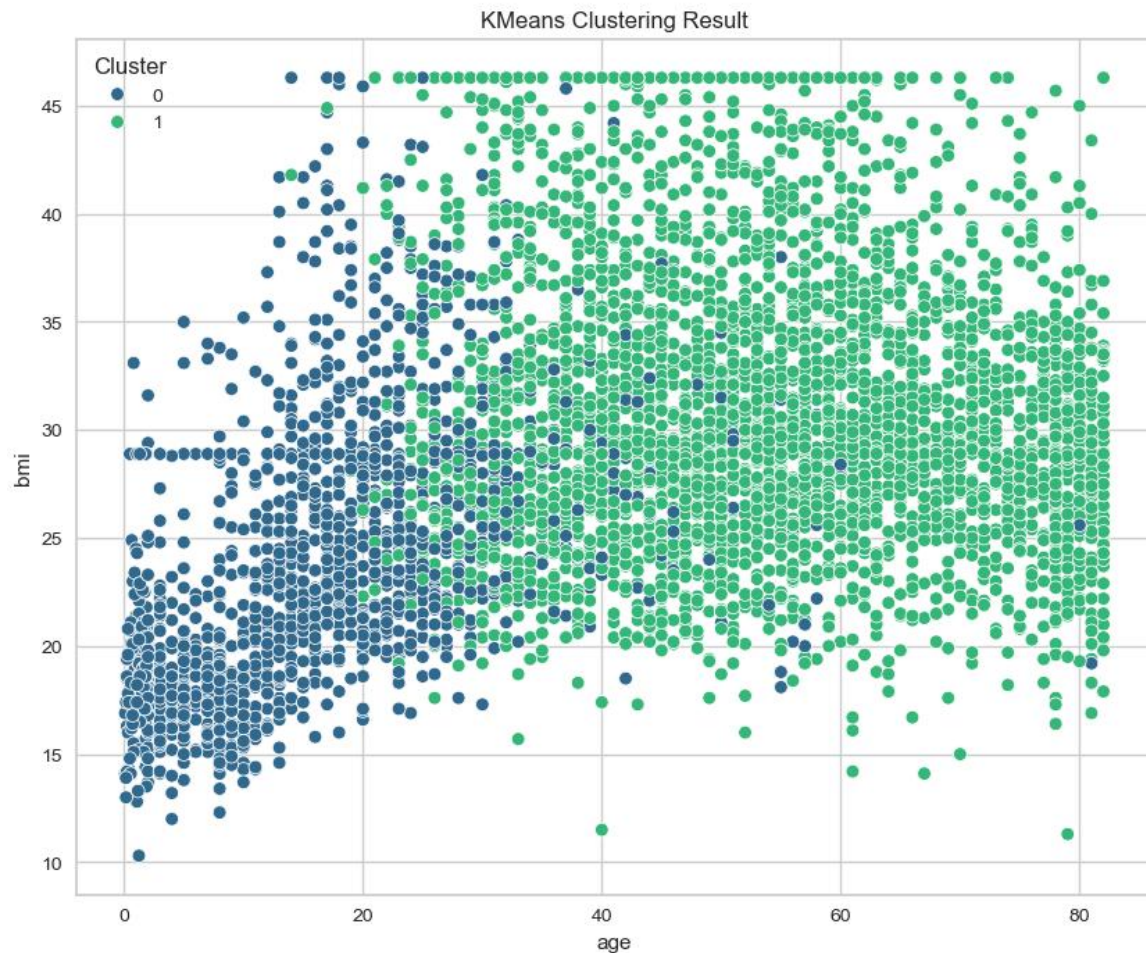
```
[-0.02041724 -0.06303319 -0.02587939 -0.2389468 -0.02739383 0.00669607
 -0.00073883 -0.03340118 -0.0111817 -0.01157947 -0.03223727]
```

Cluster 2 Mean:

```
[0.35759766 1.10399435 0.45326439 4.18503199 0.47978907 -0.11727834
 0.01294021 0.58500473 0.19584178 0.20280849 0.56461939]
```

K Means Clustering Result

```
plt.figure(figsize=(10, 8))
sns.scatterplot(x='age', y='bmi', hue='Cluster', data=df_cluster, palette='viridis')
plt.title('KMeans Clustering Result')
plt.show()
```



The scatter plot to visualize the result of the KMeans clustering analysis. It plots the relationship between two variables, age and bmi, distinguished by cluster labels. It uses the hue parameter to color-code different clusters. The viridis palette provides a gradient of colors that are visually easy to distinguish from one another. This plot is further assisted with a title, "KMeans Clustering Result," for better clarification of its purpose. It will help interpret the cluster groupings and the distribution of data points across age and bmi, hence giving an insight into the patterns or group behaviours identified by the KMeans algorithm.

Mean Shift Clustering algorithm

Feature Scaling for Standardization

```
scaler = StandardScaler()  
scaled_features = scaler.fit_transform(df_cluster.values)
```

Size of Clusters

```
cluster1.shape
```

```
(4632, 12)
```

```
cluster2.shape
```

```
(229, 12)
```

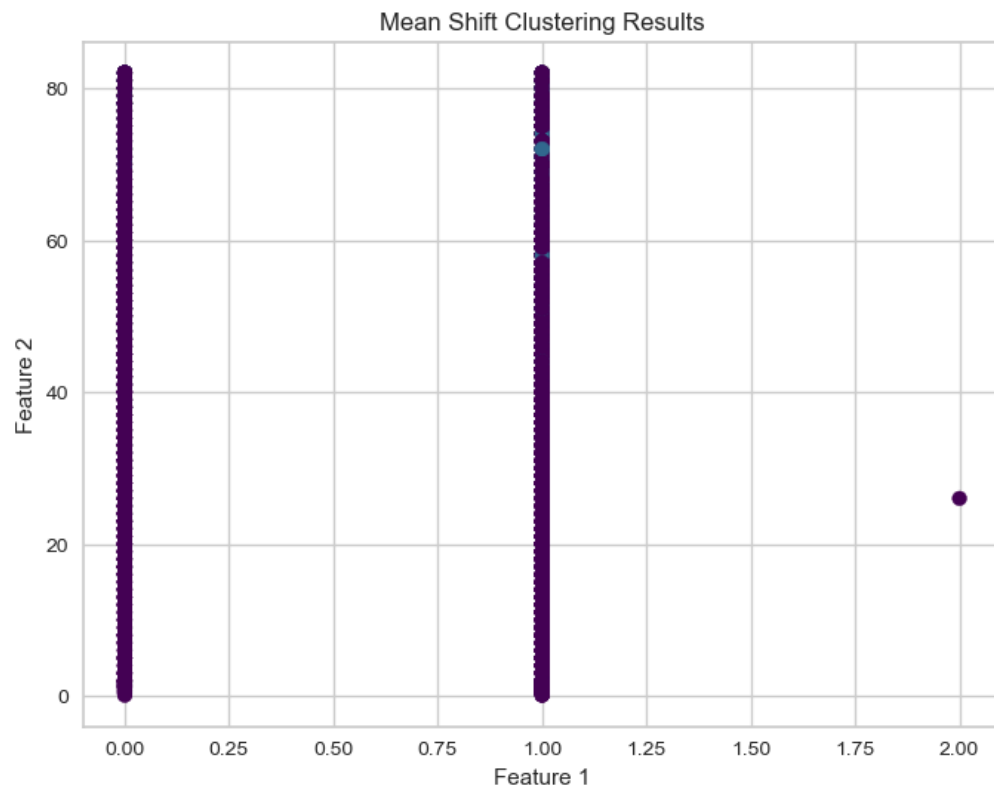
Cluster Distribution in the Dataset

```
df_cluster['cluster'].value_counts()
```

```
cluster  
0    4632  
1     229  
2     202  
3       47  
Name: count, dtype: int64
```

Visualizing Mean Shift Clustering Results

```
plt.figure(figsize=(8, 6))
plt.scatter(df_cluster.iloc[:, 0], df_cluster.iloc[:, 1], c=df_cluster['cluster'], cmap='viridis')
plt.title('Mean Shift Clustering Results')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



The following scatter plot shows the result of Mean Shift clustering. The data points are plotted against the first two features of the dataset, with each point colored according to its cluster assignment. The Viridis colormap was used to increase the visual difference between clusters. This visualization will help to interpret the spatial distribution and separation of clusters identified by the algorithm.

Comparison between K-Means Clustering and Mean Shift Clustering:

- **Number of Clusters:**
 - K-Means: Requires the number of clusters to be specified beforehand.
 - Mean Shift: Automatically determines the number of clusters based on data density.
- **Cluster Shape:**
 - K-Means: Assumes clusters are spherical.
 - Mean Shift: Works well with clusters of arbitrary shapes.
- **Cluster Centers:**
 - K-Means: Uses centroids as cluster centers.
 - Mean Shift: Identifies high-density regions as cluster centers.
- **Sensitivity to Outliers:**
 - K-Means: Sensitive to outliers, which can shift centroids.
 - Mean Shift: More robust to outliers due to its density-based approach.
- **Execution Time:**
 - K-Means: Faster and more suitable for large datasets.
 - Mean Shift: Slower and more computationally intensive.
- **Flexibility:**
 - K-Means: Works best with evenly sized clusters.
 - Mean Shift: Handles clusters of varying sizes effectively.